



SaaS Sales Funnel Dashboard

Developed using Python, Streamlit, Pandas & Plotly

Documentation

Prepared by:
Amaal Hus

Contents

1. Introduction.....	1
1.1. Stages of a SaaS (Software as a Service) Marketing Funnel.....	1
1.1.1. Awareness.....	1
1.1.2. Engagement.....	2
1.1.3. Desire.....	3
1.1.4. Conversion	4
1.1.5. Retention	4
2. Understanding the Data	6
3. Exploratory Data Analysis (EDA).....	8
3.1. Objective of the EDA	8
3.2. Tools Used.....	8
3.3. EDA Process and Key Findings using Python.....	8
3.3.1. Preliminary Overview.....	9
3.3.2. Company Size Distribution	11
3.3.3. Funnel Stage Distribution.....	12
3.3.4. Lead Source Breakdown.....	13
3.3.5. Revenue Statistics	15
3.3.6. How Fast Sales Responds to Leads.....	16
3.3.7. Sales Rep Leads Count	16
4. Dashboard Creation.....	18
4.1. Python Libraries.....	18
4.2. Dashboard Layout Design	18
4.2.1. Dashboard Configuration	18
4.2.2. Loading the Data	19
4.2.3. Sidebar Filters.....	19
4.2.4. Header	21
4.2.5. KPIs	21
4.2.6. Funnel	23

4.2.7.	Company Size using a Combined Bar Chart and Line Chart.....	24
4.2.8.	MQL-to-Demo Booking Duration	27
4.2.9.	Monthly Funnel Trend.....	29
4.3.	Viewing the Dashboard	31
4.4.	Final Dashboard Output.....	31
5.	Conclusion.....	33
6.	References	34



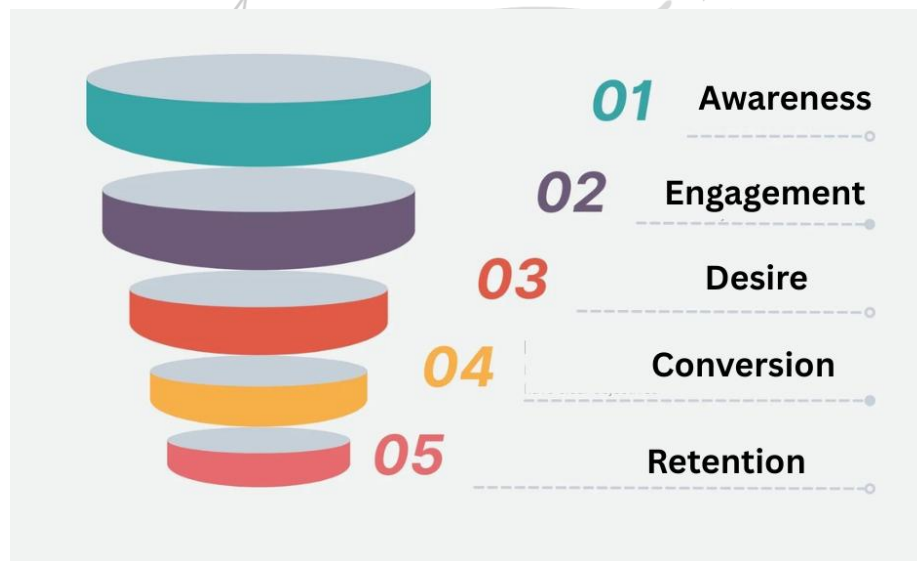
1.Introduction

This project focuses on the development of an interactive Software as a Service (SaaS) Sales Funnel Dashboard using Python. The purpose of the dashboard is to transform raw sales data into meaningful visualisations that enable users to monitor sales performance, analyse customer progression through the sales funnel, and support data-driven decision-making.

A SaaS (Software as a Service) business delivers software applications over the internet through a subscription-based model. As customers progress from initial interest to becoming paying subscribers, they pass through several stages of the sales funnel, including **Lead**, **Marketing Qualified Lead (MQL)**, **Demo Booked**, **Demo Completed**, **Trial Started**, and finally **Closed Won** or **Churned**. Understanding how customers move through these stages is essential for evaluating sales performance and identifying opportunities for improvement.

1.1. Stages of a SaaS (Software as a Service) Marketing Funnel

The stages of a Saas marketing funnel are as shown in the diagram below:



1.1.1.Awareness

During the awareness stage of a SaaS marketing funnel, potential customers are first introduced to the existence of a product or service. At this point, individuals may not yet recognize the specific challenges they are facing or understand how a particular solution can address them.

This initial touchpoint often occurs through search engine results, social media content, or online advertisements. As the primary objective of this stage is to build brand visibility and encourage initial engagement, emphasis is typically placed on strategies such as search engine optimization (SEO), as well as targeted paid search and digital advertising to drive traffic and increase exposure.

The following metrics are tracked at this stage:

- a) Website visits: the number of users who visits the website.
- b) Social media engagement: likes, comments and shares.
- c) Search engine ranking: the position of the website on the search engine results pages (SERPs) for target keywords.
- d) Bounce rate: users leaving the website without interacting with the product.

1.1.2. Engagement

Users enter the engagement stage when they return to interact with the product for a second time and sign up to receive additional content, such as an email list of newsletter. This indicates that the content has generated sufficient interest for them to explore the company and its offerings more deeply.

At this stage, it is important to capture key customer information to better understand their pain points and journey. The most commonly collected data is the customer's email address, which enables further communication and nurturing.

As users progress through engagement stage, their interaction patterns should be closely monitored. Some individuals move through the funnel more quickly than others. Highly engaged users – such as those who sign up for resources and frequently interact with content – can be segmented and targeted with more tailored communication that emphasizes how the product addresses specific problems and supports their goals.

Additionally, heatmaps can be used to track how potential customers interact with different elements on a website. This includes identifying whether users click on features such as free trial buttons or newsletter sign-ups, and how far they scroll through a page. Heatmaps provide deeper behavioral insights beyond simple page visit metrics, enabling a clearer understanding of user engagement and on-page activity.

The metrics to be tracked at this stage include:

- a) Website metrics: time on page, views and clicks (specifically where on the website users are interacting, and whether visitors are new or returning)
- b) Email signups: the amount of users who provide their email to receive content

- c) Newsletter subscriptions: the amount of visitors that sign up to receive the newsletter
- d) Social media engagement: the interactions the users have with the content on social media platforms

1.1.3.Desire

At this stage, a relationship with potential customers has already been established, and the company is viewed as a credible source of information. Interest in the product has grown, creating an opportunity to deepen engagement. At this stage, organizations typically offer a free trial or a freemium version, allowing users to explore the product and build confidence before committing to a paid plan.

This stage demonstrates the product's ability to support users in achieving their goals. By providing hands-on experience at no cost, potential customers are able to validate the product's quality and relevance, reducing uncertainty in the decision-making process.

It is important to track not only the number of users who sign up for a free trial, but also the number who actively engage with it. Activation metrics (such as whether users perform key actions or use core features) are critical indicators of genuine interest. Users who meaningfully interact with the product during the trial are more likely to progress further in the funnel and convert into paying customers, compared to those who sign up but remain inactive.

In cases where a free trial is not feasible, alternatives such as live demonstrations or customized presentations can be used to provide a similar level of product exposure and engagement.

The metrics to be tracked in the Desire stage would be:

- a) Free trial downloads: the amount of users sign up to try the product
- b) Free trial activation rate: how many users actually interact with the downloaded trial
- c) Demo requests: how many users request a personalized demo from the organisation

1.1.4. Conversion

Conversion occurs when a customer completes a key milestone, such as making a first purchase or subscribing to a paid version of a product. For instance, when a user purchases a monthly plan for a software solution, this marks a successful conversion.

At this stage, it is important to evaluate the performance through specific metrics. This includes assessing how many users who entered the marketing funnel progressed to the conversion stage, as well as monitoring trends in churn rate— whether it is increasing or decreasing over time. These indicators provide insight into the effectiveness of the funnel and highlight areas that may require improvement to drive higher conversion rates.

A deeper understanding of the customer journey can be achieved by analysing how users reached the conversion stage, as well as identifying factors that contributed to churn. Collecting user feedback plays a crucial role in this process, as it helps uncover user experiences, pain points, and reasons behind both successful conversions and drop-offs.

The metrics to be tracked in the Conversion stage are as followed:

- a) Number of purchases: how many customers pay for a product or subscription
- b) Customer acquisition cost (CAC): how much it costs the organization to get a user to convert
- c) Conversion rate: the proportion of users who make a purchase or download
- d) Churn rate: the percentage of customers who unsubscribe or stop using the product over a period of time.

1.1.5. Retention

While many traditional marketing funnels conclude at the point of initial conversion, SaaS business models depend on recurring revenue. As a result, Customer Retention stage is essential. This stage focuses on ensuring that existing customers continue their subscriptions while fostering long-term loyalty and advocacy toward the product.

Customer retention also presents opportunities for upselling and cross-selling. For instance, a SaaS product may introduce premium plans with advanced features or offer add-ons that enhance functionality, thereby increasing customer lifetime value.

Referrals and positive word of mouth from existing users play a significant role in acquiring new customers. Organisations may encourage this behaviour by offering referral incentives, exclusive promotions, or tailored content such as premium newsletters to strengthen engagement and advocacy.

If patterns of unsubscribing or increased churn are observed after a certain period, it becomes important to investigate the underlying reasons. Voice of Customer (VoC) methods (surveys, user interviews and other feedback tools) can provide valuable insights into customer pain points and overall experience, enabling targeted improvements in customer experience (CX).

Key metrics to monitor at this stage includes:

- a) Customer Retention Rate (CRR): the proportion of customers who continue their subscriptions over time
- b) Net Promoter Score (NPS): an indicator of customer loyalty and likelihood to recommend the product
- c) Customer Satisfaction (CSAT): a measure of how satisfied customers are with the product and overall experience
- d) Churn Rate: the percentage of customers who discontinue their subscription or stop using the product within a given period



2. Understanding the Data

The data used for this project is **saas_funnel_data.csv**, which consists of **15 columns (headers)** as follows:

Bil.	Name	Data Type
1	lead_id	String
2	created_date	Date
3	lead_source	String
4	company_size	String
5	industry	String
6	sales_rep	String
7	current_stage	String
8	mql_date	Date
9	demo_booked_date	Date
10	days_mql_to_demo_booked	Date
11	demo_completed_date	Date
12	trial_start_date	Date
13	closed_date	Date
14	deal_value	Float
15	churned	String

The **current_stage** column represents the stages of the Customer Relationship Management (CRM) funnel, as shown in the table below:

Bil.	CRM Stage	Definition
1	Lead	A potential customer who has shown initial interest in a product or service (e.g: Filled a form, signed up, or made an inquiry. They are at the earliest stage of the funnel.
2	MQL (Marketing Qualified Lead)	A lead that has been assessed by marketing and is considered more likely to convert based on behaviour or engagement (e.g: multiple website visits, downloading resources, engaging with campaigns)

3	Demo Booked	The lead has scheduled a product demonstration with the sales team. This showed stronger intent compared to earlier stages.
4	Demo Completed	The scheduled demo has taken place. The prospect has seen the product in action and is actively being evaluated for fir.
5	Closed Won	The deal has been successfully converted into paying customer. The sale is completed.
6	Closed Lost	The deal did not result in a sale. The opportunity is lost due to reasons such as budget constraints, competitor selection, lack of fit, or no decision.
7	Churned	A previously acquired customer who has stopped using the product or cancelled their subscription. This applies mainly to recurring revenue models like SaaS.

3.Exploratory Data Analysis (EDA)

3.1. Objective of the EDA

The objective of the exploratory data analysis (EDA) is to understand the structure, quality, and underlying patterns within the dataset before building the dashboard. This includes assessing data completeness, validating data types, and identifying any inconsistencies that may affect analysis.

Beyond data validation, the EDA aims to uncover key insights about the SaaS funnel, such as where leads drop off, which lead sources generate the highest quality prospects, how revenue is distributed, and how efficiently the sales process operates. These insights form the foundation for designing meaningful dashboard metrics and ensure that the final visualisations are accurate, relevant, and aligned with business objectives.

3.2. Tools Used

	
Visual Studio Code	Python
(src: Wikimedia Commons)	

The tool used to perform the exploratory data analysis (EDA) was Visual Studio Code, with Python as the programming language.

3.3. EDA Process and Key Findings using Python

The exploratory data analysis (EDA) was conducted using Python to systematically examine the dataset and uncover meaningful patterns related to the SaaS sales funnel. The process was designed to provide a structured understanding of how the data is

organized, while identifying trends and relationships that support further analysis and visualization.

3.3.1.Preliminary Overview

The analysis begins with an initial inspection of the dataset to determine its overall structure, including the number of records and variables. This is used to quickly understand the structure and contents of the dataset before performing deeper analysis.

Bil.	Component	Details
1.	Code	<pre> 8 # — 1. Columns and Rows — 9 print("=" * 60) 10 print("Columns and Rows") 11 print("=" * 60) 12 print(f"\nShape: {df.shape[0]} rows x {df.shape[1]} columns\n") 13 print(df.head(10).to_string()) </pre> Line 12 retrieves the dimensions of the dataset using df.shape. The expression of df.shape[0] returned the number of rows (observations), while df.shape[1] returned the numbers of columns (variables).
2.	Key Finding	<pre> ===== Columns and Rows ===== Shape: 4200 rows x 15 columns lead_id created_date lead_source company_size industry sales_rep current_stage 0 L-0001 7/24/2025 Referral Startup (1-10) Marketing James Chen Closed Won 25/2025 7/26/2025 1.0 7/27/2025 8/1/2025 8/14/2025 4977.67 False 1 L-0002 6/21/2025 Organic Search SMB (11-50) Marketing Maria Santos NaN Lead NaN NaN NaN NaN NaN NaN NaN 0.00 False 2 L-0003 5/18/2025 Paid Ads Startup (1-10) EdTech Maria Santos MQL 21/2025 NaN NaN NaN NaN NaN NaN 0.00 False 3 L-0004 6/3/2025 Referral Mid-Market (51-200) HR Tech Tom Wilson NaN Lead NaN NaN NaN NaN NaN NaN NaN 0.00 False 4 L-0005 7/8/2025 Organic Search SMB (11-50) EdTech Tom Wilson NaN Lead NaN NaN NaN NaN NaN NaN NaN 0.00 False </pre> There are a total of 4,200 rows and 15 columns.

Subsequently, the data types and completeness of each variable were evaluated. This involved identifying the format of each column and calculating the number and percentage of missing values.

Assessing these aspects was important to ensure data consistency and to highlight any potential limitations that could affect the accuracy of the analysis.

Bil.	Component	Details
1.	Code	<pre>15 # — 2. Column types & nulls — 16 print("\n" + "=" * 60) 17 print("DATA TYPES & MISSING VALUES") 18 print("=" * 60) 19 info = pd.DataFrame({ 20 "dtype": df.dtypes, 21 "non_null": df.notna().sum(), 22 "null": df.isna().sum(), 23 "null_%": (df.isna().sum() / len(df) * 100).round(1) 24 }) 25 print(f"\n{info.to_string()}")</pre> • The non_null part counts how many values in each column are not missing. It is calculated using <code>df.notna().sum()</code>, which checks every cell in a column and counts only the entries that contain actual data. This helps to show how complete each column is. • The null part counts how many missing values (NaN) exist in each column. It uses <code>df.isna().sum()</code>, which identifies empty or undefined entries and totals them per column. This is useful for spotting data quality issues or gaps in the dataset. • The info.to_string() part is used to display the info DataFrame in a clean, readable text format. Instead of showing a shortened or wrapped version of the table, <code>to_string()</code> prints the full table neatly in the console so all rows and columns can be viewed clearly.

2.

Key Finding

						dtype	non_null	null	null_%
lead_id						str	4200	0	0.0
created_date						str	4200	0	0.0
lead_source						str	4200	0	0.0
company_size						str	4200	0	0.0
industry						str	4200	0	0.0
sales_rep						str	4200	0	0.0
current_stage						str	4200	0	0.0
mql_date						str	2571	1629	38.8
demo_booked_date						str	1429	2771	66.0
days_mql_to_demo_booked						float64	1429	2771	66.0
demo_completed_date						str	1013	3187	75.9
trial_start_date						str	444	3756	89.4
closed_date						str	133	4067	96.8
deal_value						float64	4200	0	0.0
churned						bool	4200	0	0.0

3.3.2. Company Size Distribution

Company size distribution is analysed to understand how the dataset is spread across different types of organisations, such as small, medium, or large companies. This is important because company size can influence customer behaviour and conversion patterns in the sales funnel.

By identifying the distribution, it provides context for interpreting results and helps ensure more accurate analysis of performance across different business segments.

Bil.	Component	Details
1.	Code	<pre>27 # — 3. Company size breakdown — 28 print("\n" + "=" * 60) 29 print("COMPANY SIZE DISTRIBUTION") 30 print("=" * 60) 31 print(f"\n{df['company_size'].value_counts().to_string()}")</pre>

2.	Key Finding	<pre>===== COMPANY SIZE DISTRIBUTION ===== company_size Startup (1-10) 1305 SMB (11-50) 1293 Mid-Market (51-200) 1131 Enterprise (200+) 471</pre>
----	-------------	--

3.3.3.Funnel Stage Distribution

The next stage focused on examining the distribution of leads across the different stages of the SaaS funnel. By analysing the number of leads present at each stage and comparing progression between stages, it was possible to identify where significant drop-offs occurred. This provided valuable insight into potential bottlenecks within the funnel that may hinder conversion.

Bil.	Component	Details
1.	Code	<pre>33 # --- 4. Funnel stage distribution --- 34 print("\n" + "=" * 60) 35 print("WHERE ARE LEADS DROPPING OFF?") 36 print("=" * 60) 37 stages = df["current_stage"].value_counts() 38 print(f"\n{stages.to_string()}") 39 total = len(df) 40 print(f"\n Lead → MQL: {(df['mql_date'].notna().sum():>5)}/{(df['mql_date'].notna().sum()/total*100:.1f)%}") 41 print(f" MQL → Demo Booked: {(df['demo_booked_date'].notna().sum():>5)}/{(df['demo_booked_date'].notna().sum()/total*100:.1f)%}") 42 print(f" Demo Booked → Done: {(df['demo_completed_date'].notna().sum():>5)}/{(df['demo_completed_date'].notna().sum()/total*100:.1f)%}") 43 print(f" Demo → Trial: {(df['trial_start_date'].notna().sum():>5)}/{(df['trial_start_date'].notna().sum()/total*100:.1f)%}") 44 print(f" Trial → Closed Won: {(len(df[df['current_stage']=='Closed Won']):>5)}/{(len(df[df['current_stage']=='Closed Won']/total*100:.1f)%}")</pre> Line 38 prints the funnel stage counts in a clean table format. • stages is a pandas Series • to_string() ensures the full output is displayed neatly in text form without truncation For Line 40 until Line 44, each of the line calculates: • How many users reached a stage • What percentage of total users that represents Eg: <code>{df['trial_start_date'].notna().sum():>5}</code> • df['trial_start_date'] → selects the column • .notna() → checks each row:

		○ True if there is a value (not missing) ○ False if it is missing (NaN) • .sum() → counts how many True values there are. • :>5 is simply the formatting instruction where it means to align the number to the right & use a minimum width of 5 characters
2.	Key Finding	<pre> ===== WHERE ARE LEADS DROPPING OFF? ===== current_stage Lead 1629 MQL 1142 Demo Completed 569 Demo Booked 416 Closed Lost 311 Closed Won 110 Churned 23 Lead → MQL: 2571 (61.2%) MQL → Demo Booked: 1429 (34.0%) Demo Booked → Done: 1013 (24.1%) Demo → Trial: 444 (10.6%) Trial → Closed Won: 110 (2.6%) </pre>

3.3.4. Lead Source Breakdown

Additionally, the distribution of lead sources was analysed to determine which channels contributed the highest number of leads. This helped in understanding the effectiveness of various marketing efforts.

Bil.	Component	Details
1.	Code	<pre> 46 # — 5. Lead source breakdown — 47 print("\n" + "=" * 60) 48 print("LEAD SOURCES – WHICH ONE BRINGS THE MOST?") 49 print("=" * 60) 50 print(f"\n{df['lead_source'].value_counts().to_string()}") 51 </pre>

2.	Key Finding	<pre>===== LEAD SOURCES – WHICH ONE BRINGS THE MOST? ===== lead_source Paid Ads 1498 Organic Search 1078 LinkedIn 761 Cold Outreach 468 Referral 395</pre>
----	-------------	--

A cross-tabulation between lead sources and final funnel stages was also conducted to evaluate the quality of leads generated by each channel, allowing for a more comprehensive assessment of both volume and conversion performance.

Bil.	Component	Details
1.	Code	<pre>52 # — 6. Quick cross-tab: source x stage — 53 print("\n" + "=" * 60) 54 print("CROSS-TAB: SOURCE x FINAL STAGE") 55 print("=" * 60) 56 ct = pd.crosstab(df["lead_source"], df["current_stage"]) 57 print(f"\n{ct.to_string()}")</pre> Using the function crosstab, two or more categorical variables can be summarized into a table of counts. Thus, for Line 56, it means “Count how many records exist for each combination of lead source and CRM stage”, where: • df["lead_source"] is the row variable; and • df["current_stage"] is the column variable.
2.	Key Finding	

		<pre>===== CROSS-TAB: SOURCE x FINAL STAGE ===== current_stage Churned Closed Lost Closed Won Demo Booked Demo Completed Lead MQL lead_source Cold Outreach 0 9 1 26 17 334 81 LinkedIn 1 59 12 85 90 299 215 Organic Search 11 97 33 114 205 315 303 Paid Ads 2 60 7 166 160 632 471 Referral 9 86 57 25 97 49 72</pre>
--	--	--

3.3.5. Revenue Statistics

Revenue-related metrics were examined by focusing on successfully closed deals. Key indicators such as total revenue, average deal size, median value, and the highest deal recorded were calculated to evaluate financial outcomes.

This provided a direct connection between funnel performance and business results.

Bil.	Component	Details
1.	Code	<pre>59 # — 7. Revenue stats — 60 print("\n" + "=" * 60) 61 print("REVENUE (closed deals only)") 62 print("=" * 60) 63 closed = df[df["deal_value"] > 0] 64 print(f"\n Total deals closed: {len(closed)}") 65 print(f" Total revenue: \${closed['deal_value'].sum():,.0f}") 66 print(f" Avg deal size: \${closed['deal_value'].mean():,.0f}") 67 print(f" Median deal size: \${closed['deal_value'].median():,.0f}") 68 print(f" Largest deal: \${closed['deal_value'].max():,.0f}")</pre>
2.	Key Finding	<pre>===== REVENUE (closed deals only) ===== Total deals closed: 133 Total revenue: \$3,914,872 Avg deal size: \$29,435 Median deal size: \$19,575 Largest deal: \$115,647</pre>

3.3.6.How Fast Sales Responds to Leads

This metric evaluates the speed and efficiency of the sales team in responding to qualified leads and moving them through the sales funnel. It measures the number of days taken for a Marketing Qualified Lead (MQL) to progress to a booked product demonstration.

Bil.	Component	Details
1.	Code	<pre>70 # — 8. Speed to demo — 71 print("\n" + "=" * 60) 72 print("DAYS FROM MQL → DEMO BOOKING") 73 print("=" * 60) 74 speed = df["days_mql_to_demo_booked"].dropna() 75 print(f"\n Count: {len(speed)}") 76 print(f" Mean: {speed.mean():.1f} days") 77 print(f" Median: {speed.median():.1f} days") 78 print(f" Max: {speed.max():.0f} days")</pre> • speed.mean() = calculates the average value of all numbers in the speed variable. • The :.1f is a format specifier in Python's f-strings ○ It means round of to the nearest 1 decimal place
2.	Key Finding	<pre>===== DAYS FROM MQL → DEMO BOOKING ===== Count: 1429 Mean: 5.3 days Median: 4.0 days Max: 14 days</pre>

3.3.7.Sales Rep Leads Count

This is used to measure the numbers of leads assigned to or handled by each sales representative.

This metric provides visibility into how leads are distributed across the sales team and helps assess whether workload allocation is balanced and appropriate.

Bil.	Component	Details
1.	Code	<pre> 80 # — 9. Sales rep lead count — 81 print("\n" + "=" * 60) 82 print("SALES REP – LEAD DISTRIBUTION") 83 print("=" * 60) 84 print(f"\n{df['sales_rep'].value_counts().to_string()}") </pre>
2.	Key Finding	<pre> ===== SALES REP – LEAD DISTRIBUTION ===== sales_rep James Chen 883 Priya Patel 848 Maria Santos 824 Alex Thompson 823 Tom Wilson 822 </pre>



4. Dashboard Creation

4.1. Python Libraries

The Python Libraries used for this dashboard is as followed:

Bil.	Library	Details
1.	Streamlit	Creates the web dashboard.
2.	Pandas	Loads and manipulates data.
3.	Plotly Express	Creates statistical charts.
4.	Plotly Graph Objects	Creates advanced visualisations.

```
2 import streamlit as st
3 import pandas as pd
4 import plotly.express as px
5 import plotly.graph_objects as go
```

4.2. Dashboard Layout Design

4.2.1. Dashboard Configuration

Function used to create the dashboard is **st.set_page_config()**; this is placed at the beginning of the script.

```
8 # — Config: Creating the Dashboard —
9 st.set_page_config(page_title = 'SaaS Funnel Dashboard',
10                  page_icon='🔍',
11                  layout='wide')
```

4.2.2.Loading the Data

```
13 # — Load data —
14 @st.cache_data
15 def load_data():
16     df = pd.read_csv("saas_funnel_data.csv", parse_dates=[
17         "created_date", "mql_date", "demo_booked_date",
18         "demo_completed_date", "trial_start_date", "closed_date"
19     ])
20     df["created_month"] = df["created_date"].dt.to_period("M").astype(str)
21     return df
22
23 df = load_data()
```

Line 16 – Line 18 use the function **pd.read_csv** to import the dataset into a Pandas DataFrame.

The function **parse_dates** automatically converts the specified columns into datetime objects when the CSV file is loaded. This enables data-related operations such as filtering records by date, calculating time intervals, grouping data by month, and generating time-series visualisations.

Whereas on Line 20, a new column ["created_month"] is created by extracting the year and month from the column created_date. The values are converted into a string format (e.g: 10-2025) so they can be used for monthly grouping and trend analysis in the dashboard.

4.2.3.Sidebar Filters

This section creates an interactive sidebar that allows users to filter the dashboard data based on date range, company size, and sales representative. The selected filters are applied to the dataset, ensuring that all key performance indicators (KPIs), tables, and visualisations update dynamically according to the user's selections.

```
25 # — Sidebar Filters —
26 st.sidebar.title("🔍 Filters")
27
28 date_range = st.sidebar.date_input(
29     "Date range",
30     value=(df["created_date"].min(), df["created_date"].max()),
31     min_value=df["created_date"].min(),
32     max_value=df["created_date"].max(),
33 )
34
35 sizes = st.sidebar.multiselect("Company Size", df["company_size"].unique().tolist(), default=df["company_size"].unique().tolist())
36 reps = st.sidebar.multiselect("Sales Rep", df["sales_rep"].unique().tolist(), default=df["sales_rep"].unique().tolist())
37
```

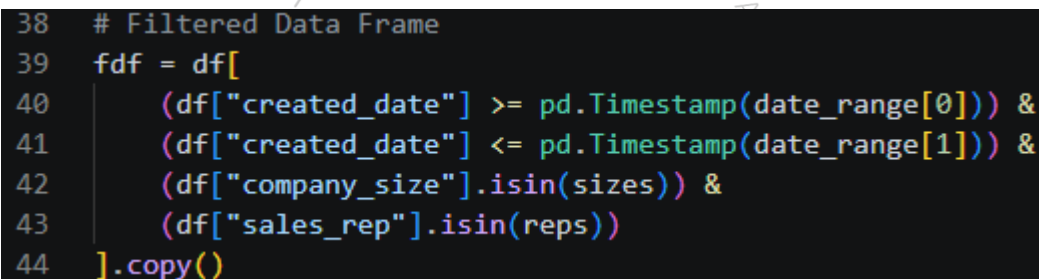
The **st.sidebar.date_input** function creates a date range selector that enables users to filter records based on the created_date column.

- The default value is set to the earliest and latest dates available in the dataset, ensuring that all records are displayed when the dashboard is first loaded.
- The minimum and maximum selectable dates are restricted to the available dates in the dataset, preventing users from selecting dates outside the valid range.

Line 35 & 36

The **st.sidebar.multiselect()** function creates a drop-down list that allows users to select one or more categories. This option is obtained by **.unique().tolist()**.

Said function retrieves all unique values from the dataset and converts them into a list. By default, all company sizes are selected so that the entire dataset is displayed when the dashboard is first opened.



```

38 # Filtered Data Frame
39 fdf = df[
40     (df["created_date"] >= pd.Timestamp(date_range[0])) &
41     (df["created_date"] <= pd.Timestamp(date_range[1])) &
42     (df["company_size"].isin(sizes)) &
43     (df["sales_rep"].isin(reps))
44 ].copy()

```

As for the filtered DataFrame, the selected filter criteria are applied to the original dataset (df) to create a new DataFrame named **fdf**. This DataFrame contains only the records that satisfy the user's selected date range, company size, and sales representative filters.

The filtering conditions are as follows:

- The **created_date** must fall within the selected date range.
- The **company_size** must match one of the selected company size categories.
- The **sales_rep** must match one of the selected sales representative.

The **.isin()** method checks whether each value belongs to the list of selected options, while **pd.Timestamp()** converts the selected dates into a compatible datetime format for comparison.

Finally, the **.copy()** method creates an independent copy of the filtered dataset. This prevents unintended modifications to the original DataFrame and allows subsequent data processing and visualisations to use the filtered data safely.

4.2.4.Header

```
47 # — Header + KPIs —
48 st.title("📊 SaaS Funnel Dashboard")
49 st.caption(f"Analyzing {len(fdf):,} leads from {date_range[0]} to {date_range[1]}")
```

The **st.caption()** function displayed a brief description beneath the dashboard title which is generated dynamically using an f-string, which inserts values from the filtered dataset into the displayed text.

The caption includes:

Bil.	Function	Details
1.	Len(fdf)	Returns the total number of leads contained in the filtered DataFrame.
2.	:,	Formats the number with comma separators for improved readability (eg: 1,250 instead of 1250).
3.	Date_range[0]	Displays the start date selected by the user.
4.	Date_range[1]	Displays the end date selected by the user.

4.2.5.KPIs

```
51 won = fdf[fdf["current_stage"] == "Closed Won"]
52 churned = fdf[fdf["current_stage"] == "Churned"]
53 closed_total = len(won) + len(churned)
```

The filtered dataset is further divided into two subsets based on the value of the `current_stage` column:

Bil.	Column	Details
1.	won	Contains all the records where the sales opportunity has been successfully converted into a customer (Closed Won)
2.	churned	Contains all records where a customer was acquired but later discontinued the service (Churned)

The variable `closed_total` is then calculated by adding the number of Closed Won and Churned records. This represents the total number of leads that reached the final stage of the sales pipeline and is used to calculate the dashboard's performance metrics.

```
55 k1, k2, k3, k4, k5 = st.columns(5)
56 k1.metric("Total Leads", f"{len(fdf):,}") # F Strings handle the formatting
57 k2.metric("Closed Won", f"{len(won):,}")
58 k3.metric("Revenue", f"${fdf['deal_value'].sum():,.0f}")
59 k4.metric("Win Rate", f"{closed_total / len(fdf) * 100:.1f}%" if len(fdf) else "0%")
60 k5.metric("Churn Rate", f"{len(churned) / closed_total * 100:.1f}%" if closed_total else "0%")
61
62 st.divider()
```

The KPI metrics is divided into five, which are:

- Total Leads
- Closed Won
- Revenue
- Win Rate
- Churn Rate

Bil.	KPI	Details
1.	k1.metric("Total Leads", f"{len(fdf):,}")	<u>Line 56</u> • Displays the total number of leads contained in the filtered dataset. • The <code>len()</code> function counts the records, while the f-string formats the number with comma separators for improved readability.
2.	k2.metric("Closed Won", f"{len(won):,}")	<u>Line 57</u> Displays the number of leads that have successfully reached the Closed Won stage.
3.	k3.metric("Revenue", f"\${fdf['deal_value'].sum():,.0f}")	<u>Line 58</u> • Calculates and displays the total revenue by summing the values in the <code>deal_value</code> column. • Output is formatted as currency with comma separators and no decimal places.

4.	k4.metric("Win Rate", f"{closed_total / len(fdf) * 100:.1f}%" if len(fdf) else "0%")	<u>Line 59</u> - Win Rate represents the percentage of leads that progressed to the final stage of the sales pipeline - Formula: $\text{Win Rate (\%)} = (\text{Total Closed Deals} \div \text{Total Leads}) \times 100$ - An if statement checks whether the filtered dataset contains any records before performing the calculation – if dataset is empty, the dashboard displays 0%, preventing a division-by-zero error.
5.	k5.metric("Churn Rate", f"{len(churned) / closed_total * 100:.1f}%" if closed_total else "0%")	<u>Line 60</u> - Churn Rate measures the percentage of closed customers who have churned - Formula: $\text{Churn Rate (\%)} = (\text{Number of Churned Customers} \div \text{Total Closed Deals}) \times 100$

4.2.6. Funnel

Funnel chart is used to illustrate how leads progress through each stage of the sales pipeline. A funnel chart is particularly suitable because it visually represents the gradual reduction in the number of leads as they move from the initial stage to the conversion stage.

```

65 # — 1) Funnel —
66 st.subheader(" ▼ Full Funnel Breakdown")
67
68 funnel_stages = ["Lead", "MQL", "Demo Booked", "Demo Completed", "Trial Started", "Closed Won"]
69 funnel_counts = [
70     len(fdf),
71     fdf["mql_date"].notna().sum(),
72     fdf["demo_booked_date"].notna().sum(),
73     fdf["demo_completed_date"].notna().sum(),
74     fdf["trial_start_date"].notna().sum(),
75     closed_total,
76 ]

```

Line 71 – 74

The `.notna().sum()` method counts the number of rows where the corresponding date field is not empty (NaN), indicating that the lead has successfully reached that stage.

```
77
78 fig_funnel = go.Figure(go.Funnel(
79     y=funnel_stages, x=funnel_counts,
80     textinfo="value+percent previous",
81     marker=dict(color=["#4C78A8", "#54A24B", "#E45756", "#F58518", "#72B7B2", "#FF9DA6"]),
82 ))
83 fig_funnel.update_layout(height=420, margin=dict(t=10, b=10))
84 st.plotly_chart(fig_funnel, use_container_width=True)
```

Line 78

The `go.Funnel()` function from Plotly Graph Objects is used to create the funnel chart.

Line 80

The parameter `textinfo="value+percent previous"` displays information on each section of the funnel:

- Total number of leads at that stage;
- Percentage retained compared with the previous stage.

4.2.7. Company Size using a Combined Bar Chart and Line Chart

Note: `px.bar` builds the bars while `add_scatter` builds the line.

```
86 # — 2) Company Size —
87 st.subheader("📊 Company Size: Volume vs. Close Rate")
88
89 size_order = ["Startup (1-10)", "SMB (11-50)", "Mid-Market (51-200)", "Enterprise (200+)"]
90
91 sz = fdf.groupby("company_size").agg(
92     leads=("lead_id", "count"),
93     wins=("current_stage", lambda x: x.isin(["Closed Won", "Churned"]).sum()),
94 ).reindex(size_order)
95 sz["win_rate"] = (sz["wins"] / sz["leads"] * 100).round(1)
96
```

Line 91

`groupby()` function groups the filtered dataset according to the values in the `company_size` column; each company size category is treated as a separate group, allowing summary statistics to be calculated for each category independently.

The `agg()` function is used to calculate multiple summary statistics for each company size group.

Two metrics are generated:

- Leads: counts the total number of leads within each company size category by counting the values in the lead_id column
- Wins: counts the number of leads that reached the final stage of the sales pipeline. The lambda function checks whether the current_stage is either Closed Won or Churned, and counts the matching records.

Using a Lambda function

A lambda function is an anonymous function used to perform a simple operation without defining a separate function using def. It is commonly used with functions such as **groupby()** and **agg()** when a custom calculation is required.

In this case, the lambda function is applied to the current_stage column for each company size group.

- **x** represents all the values in the current_stage column for a single company size category.
- The **.isin(["Closed Won", "Churned"])** method checks whether each value in **x** matches either "Closed Won" or "Churned".
- It returns a Boolean series, where matching values are True and non-matching values are False.

The **.sum()** function then counts the number of True values. Since Python treats True as 1 and False as 0, summing the Boolean values gives the total number of records that are either Closed Won or Churned.

Therefore, the lambda function calculates the total number of leads that have reached the final stage of the sales pipeline for each company size category.

4.2.7.1. *Creating the Bar Chart*

```
97 fig_sz = px.bar(sz, x=sz.index,
98                 y="leads",
99                 text="leads",
100                title="Lead Volume & Win Rate by Company Size")
```

This creates the base chart using a bar chart:

- **x=sz.index** - Company size categories (Startup, SMB, etc_
- **y="leads"** - Number of leads in each category
- **text="leads"** - displays the numeric value on top of each bar

4.2.7.2. *Adding the Line Chart for Win Rate*

```

101 fig_sz.add_scatter(x=sz.index, y=sz["win_rate"],
102                    mode="lines+markers+text",
103                    text=sz["win_rate"].astype(str) + "%",
104                    textposition="top center",
105                    name="Win Rate %",
106                    yaxis="y2",
107                    line=dict(color="#E45756", width=3),
108                    marker=dict(size=10))

```

This overlays a line chart on top of the bar chart to show the win rate (%).

- **y=sz["win_rate"]** - Win rate values for each company size.
- **mode="lines+markers+text"** – Displays a line connecting points, markers at each data point and text labels showing percentages.
- **text=sz["win_rate"].astype(str) + "%"** - Adds percentage symbols to labels
- **yaxis="y2"** - Uses a **second y-axis** because win rate (%) is on a different scale from lead counts
- **marker=dict(size=10)** - Makes data points more visible

4.2.7.3. *Configuring the Layout*

```

109 fig_sz.update_layout(
110     yaxis2=dict(title="Win Rate (%)",
111                 overlaying="y",
112                 side="right"),
113     height=380,
114     showlegend=True,
115     legend=dict(orientation="h", yanchor="bottom", y=1.02),
116 )
117 st.plotly_chart(fig_sz, use_container_width=True)

```

Line 110 – 112

- **yaxis2** - Creates a second vertical axis on the right side for win rate (%)
- **overlaying="y"** - Shares the same chart space as the bar chart
- **side="right"** - Places the win rate axis on the right side

Line 117

This line renders the chart inside the Streamlit dashboard. By using **use_container_width=True**, it makes the chart responsive to full page width.

4.2.8.MQL-to-Demo Booking Duration

4.2.8.1. *Classifying the Speed*

```
120 # — 3) Speed to Demo —
121 st.subheader("🌟 MQL-to-Demo Booking Duration")
122
123 speed_df = fdf[df["days_mql_to_demo_booked"].notna()].copy()
124 speed_df["speed"] = speed_df["days_mql_to_demo_booked"].apply(
125     lambda x: "Fast (1-4 days)" if x <= 4 else "Slow (5+ days)"
126 )
```

Bil.	Line	Details
1.	123	• .notna() – checks if the value is NOT empty (not NaN) • .copy() – makes a separate dataset so the df is not accidentally modified. • The column fdf["days_mql_to_demo_booked"] contains number of days.
2	124 - 125	• For column "days_mql_to_demo_booked", the function apply() goes through each row one by one. • Lambda x – the x represents a single value

4.2.8.2. *Creating the Summary Table*

```

134 # Summary table
135 summary = speed_df.groupby("speed").agg(
136     Leads=("lead_id", "count"),
137     Wins=("current_stage",
138         lambda x: x.isin(["Closed Won", "Churned"]).sum())
139 )
140
141 summary["Win Rate %"] = (
142     summary["Wins"] / summary["Leads"] * 100
143 ).round(1)
144
145 st.subheader("Speed Performance Summary")
146 st.dataframe(summary, use_container_width=True)

```

Win Rate % is not performed inside the **.agg()** function as:

- **.agg()** is mainly for simple aggregations (count, sum, mean)
- Win Rate is a calculated ratio, not a direct aggregation

Bil.	Line	Details
1.	138	lambda x: x.isin(["Closed Won", "Churned"]).sum() • Used to count how many rows meet a condition inside each group • x represents the current_stage column for one group • x.isin(["Closed Won", "Churned"]) checks each value (x) so it becomes a True/False list. • .sum() – count the True values
2	141 - 142	summary["Win Rate %"] = (summary["Wins"] / summary["Leads"] * 100).round(1) • This created a new column called Win Rate %

4.2.8.3. *Creating the Histogram*

```

148 # Histogram
149 fig_spd = px.histogram(
150     speed_df,
151     x="days_mql_to_demo_booked",
152     color="current_stage",
153     nbins=14,
154     title="Distribution of Days MQL → Demo Booking",
155     barmode="stack",
156 )
157
158 fig_spd.update_layout(
159     height=400,
160     xaxis_title="Days from MQL to Demo Booking",
161     yaxis_title="Number of Leads"
162 )
163
164 st.plotly_chart(fig_spd, use_container_width=True)
165

```

The histogram is created by using **px.histogram** along with the same template; variables, dataset and any customization can be modified.

4.2.9. Monthly Funnel Trend

For this section, the filtered dataset is grouped by the month in which each lead was created, and additional metrics are calculated to support the monthly trend visualization.

```

166 # — 4) Monthly Trend —
167 st.subheader("📊 Monthly Funnel Trend")
168
169 monthly = fdf.groupby("created_month").agg(
170     leads=("lead_id", "count"),
171     wins=("current_stage", lambda x: x.isin(["Closed Won", "Churned"]).sum()),
172 ).reset_index()
173 monthly["win_rate"] = (monthly["wins"] / monthly["leads"] * 100).round(1)
174 monthly["period"] = monthly["created_month"].apply(
175     lambda m: "Q4" if any(q in m for q in ["2025-10", "2025-11", "2025-12"]) else "Other"
176 )

```

The filtered dataset (fdf) is grouped according to the created_month column using the **groupby()** function.

The **agg()** function then calculates two summary statistics for each month:

- Leads – counts the total number of leads created during the month.

- Wins – counts the number of leads that reached the final stage (Closed Won or Churned) using a lambda function.

Finally, **reset_index()** converts the grouped month values from an index back into a normal column, making the DataFrame suitable for plotting.

Bil.	Line	Details
1.	174 - 175	monthly["period"] = monthly["created_month"].apply(lambda m: "Q4" if any(q in m for q in ["2025-10", "2025-11", "2025-12"]) else "Other" • This creates a new column called period, which classifies each month into one of two categories: ○ Q4 – October, November and December 2025 ○ Other – all remaining months • The .apply() function processes each month individually, while the any() function checks whether the month matches any of the specified Q4 months; if a match is found, the month is labelled “Q4”, otherwise it is labelled as “Other”. • Classification is later used to apply different colours to the bars in the monthly trend chart, allowing Q4 to be highlighted for easier comparison.

```

178 fig_trend = px.bar(monthly, x="created_month",
179                    y="leads",
180                    color="period",
181                    color_discrete_map={"Q4": "#F6C343", "Other": "#4C78A8"},
182                    title="Leads per Month (Q4 highlighted)")
183
184 fig_trend.add_scatter(x=monthly["created_month"],
185                      y=monthly["win_rate"],
186                      mode="lines+markers",
187                      name="Win Rate %",
188                      yaxis="y2",
189                      line=dict(color="#E45756", width=3),
190                      marker=dict(size=8))

```

The next part of the code creates a combination chart consisting of a bar chart and a line chart. The bar chart displays the number of leads created each month, while the line chart shows the corresponding monthly win rate.

This is achieved by using built-in functions by the Plotly library.

```

192 fig_trend.update_layout(
193     yaxis2=dict(title="Win Rate (%)",
194                 overlaying="y",
195                 side="right"),
196     height=400, legend=dict(orientation="h",
197                             yanchor="bottom",
198                             y=1.02),
199 )
200 st.plotly_chart(fig_trend, use_container_width=True)

```

The **update_layout()** function modifies the overall appearance and formatting of the chart after it has been created instead of using the Plotly's default settings.

4.3. Viewing the Dashboard

To launch the Streamlit dashboard on a local server, open the terminal and execute the following command:

```

PS C:\Users\SUMMHEREIN\Documents\Projects\Python\01 - SaaS Funnel Dashboard> streamlit run 02_SaaSDashboard.py
2026-07-06 10:58:45.745 Uvicorn server started on 0.0.0.0:8501

```

4.4. Final Dashboard Output

The completed dashboard integrates multiple interactive visualisations and key performance indicators (KPIs) into a single interface, providing users with an overview of the SaaS sales funnel. The dashboard enables users to monitor sales performance, analyse conversion trends and identify potential bottlenecks throughout the customer acquisition process.

Users can interact with the dashboard by applying filters such as Date Range, Company Size and Sales Representative, allowing the visualisations and metrics to update dynamically based on the selected criteria.

Filters

Date range

2025/04/01 - 2026/03/31

Company Size

Startup (1-10)

SMB (11-50)

Mid-Market (51-200)

Enterprise (200+)

Sales Rep

James Chen

Maria Santos

Tom Wilson

Priya Patel

Alex Thompson

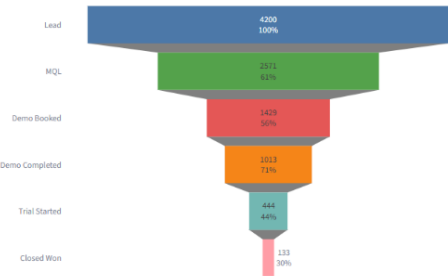
Deploy

SaaS Funnel Dashboard

Analyzing 4,200 leads from 2025-04-01 to 2026-03-31

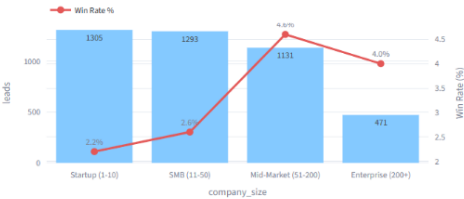
Total Leads	Closed Won	Revenue	Win Rate	Churn Rate
4,200	110	\$3,91...	3.2%	17.3%

Full Funnel Breakdown



Company Size: Volume vs. Close Rate

Lead Volume & Win Rate by Company Size

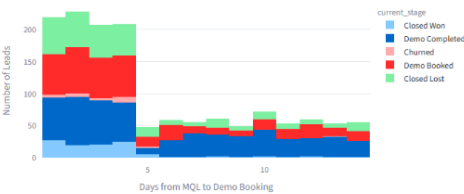


MQL-to-Demo Booking Duration

Speed Performance Summary

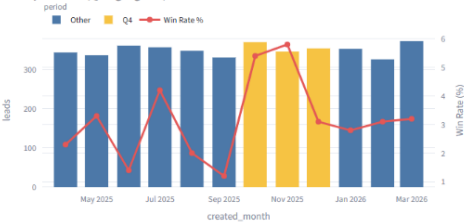
speed	Leads	Wins	Win Rate %
Fast (1-4 days)	862	114	13.2
Slow (5+ days)	567	19	3.4

Distribution of Days MQL → Demo Booking



Monthly Funnel Trend

Leads per Month (Q4 highlighted)



5. Conclusion

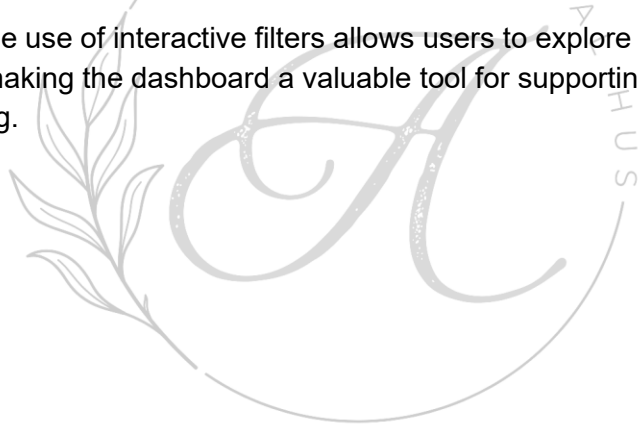
The SaaS Funnel Dashboard successfully transforms raw sales data into an interactive and informative visualisation platform. By combining KPIs, charts, and filtering capabilities, the dashboard enables users to monitor sales performance, identify trends, evaluate conversion efficiency, and support data-driven decision-making.

The key features of the final dashboard includes:

- Interactive filtering using Streamlit sidebar widgets.
- Automatic KPI calculations based on filtered data.
- Multiple Plotly visualisations for analysic sales performance.
- Responsive dashboard layout that adjusts to different screen sizes.
- Dynamic updates without requiring manual recalculation.

This project demonstrates how Python can be used to develop an interactive business intelligence dashboard using Streamlit, Pandas, and Plotly. By integrating data processing, performance metrics, and interactive visualisations into a single application, the dashboard provides an efficient way to monitor sales funnel performance.

Furthermore, the use of interactive filters allows users to explore the data from different perspectives, making the dashboard a valuable tool for supporting business analysis and decision-making.



6. References

<https://contentsquare.com/guides/marketing-funnel/saas/>

